

## Experiment 6: Linear Regression (Predicting BMI on the Pima Indians Diabetes Dataset)

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

import warnings
warnings.filterwarnings("ignore")
```

### Load Dataset

```
url =
"https://raw.githubusercontent.com/jbrownlee/Datasets/master/pima-
indians-diabetes.data.csv"

columns = [
    "Pregnancies",
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI",
    "DiabetesPedigreeFunction",
    "Age",
    "Outcome"
]

df = pd.read_csv(url, names=columns)
```

### Dataset Information

```
print("=" * 60)
print("PIMA INDIANS DIABETES DATASET")
print("=" * 60)

print("\nFirst 5 records:")
```

```

print(df.head())

print("\nDataset Shape:")
print(df.shape)

print("\nDataset Information:")
df.info()

print("\nStatistical Summary:")
print(df.describe())

print("\nNumber of Zero Values in Each Column:")
print((df == 0).sum())

```

## Data Preprocessing

```

# In these columns, zero is considered an invalid/missing value

invalid_columns = [
    "Glucose",
    "BloodPressure",
    "SkinThickness",
    "Insulin",
    "BMI"
]

# Replace zero values with NaN

df[invalid_columns] = df[invalid_columns].replace(0, np.nan)

print("\nMissing Values After Replacing Invalid Zeros:")
print(df.isnull().sum())

# Replace missing values with median

for column in invalid_columns:
    df[column] = df[column].fillna(df[column].median())

print("\nMissing Values After Median Imputation:")
print(df.isnull().sum())

```

## Select Features And Target

```

# BMI is selected as the continuous target variable

X = df.drop("BMI", axis=1)
y = df["BMI"]

print("\nPredictor Variables:")

```

```
print(list(X.columns))

print("\nTarget Variable:")
print("BMI")
```

## Train-Test Split

```
X_train, X_test, y_train, y_test = train_test_split(
    X,
    y,
    test_size=0.20,
    random_state=42
)

print("\nTraining Data Size:", X_train.shape)
print("Testing Data Size:", X_test.shape)
```

## Train Linear Regression Model

```
model = LinearRegression()

model.fit(X_train, y_train)

print("\nLinear Regression Model Trained Successfully.")
```

## Regression Coefficients

```
print("\n" + "=" * 60)
print("REGRESSION COEFFICIENTS")
print("=" * 60)

coefficients = pd.DataFrame({
    "Feature": X.columns,
    "Coefficient": model.coef_
})

print(coefficients)

print("\nIntercept:", model.intercept_)
```

## Generate Predictions

```
y_pred = model.predict(X_test)

print("\nFirst 10 Predictions:")

prediction_table = pd.DataFrame({
    "Actual BMI": y_test.values,
    "Predicted BMI": y_pred
```

```
})  
  
print(prediction_table.head(10))
```

## Model Performance Evaluation

```
mae = mean_absolute_error(y_test, y_pred)  
mse = mean_squared_error(y_test, y_pred)  
rmse = np.sqrt(mse)  
r2 = r2_score(y_test, y_pred)  
  
print("\n" + "=" * 60)  
print("REGRESSION MODEL PERFORMANCE")  
print("=" * 60)  
  
print(f"Mean Absolute Error (MAE) : {mae:.4f}")  
print(f"Mean Squared Error (MSE) : {mse:.4f}")  
print(f"Root Mean Squared Error : {rmse:.4f}")  
print(f"R2 Score : {r2:.4f}")
```

## Residual Analysis

```
residuals = y_test - y_pred  
  
print("\n" + "=" * 60)  
print("RESIDUAL ANALYSIS")  
print("=" * 60)  
  
print("\nFirst 10 Residuals:")  
print(residuals.head(10))  
  
print("\nMean Residual:", residuals.mean())
```

## Actual Vs Predicted BMI Plot

```
plt.figure(figsize=(8, 6))  
  
plt.scatter(  
    y_test,  
    y_pred,  
    color="blue",  
    alpha=0.7  
)  
  
# Perfect prediction line  
  
plt.plot(
```

```

    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()],
    color="red",
    linewidth=2
)

plt.xlabel("Actual BMI")
plt.ylabel("Predicted BMI")

plt.title("Actual vs Predicted BMI")

plt.grid(alpha=0.3)

plt.show()

```

## Residual Plot

```

plt.figure(figsize=(8, 6))

plt.scatter(
    y_pred,
    residuals,
    color="purple",
    alpha=0.7
)

# Zero residual reference line

plt.axhline(
    y=0,
    color="red",
    linestyle="--",
    linewidth=2
)

plt.xlabel("Predicted BMI")
plt.ylabel("Residuals")

plt.title("Residual Plot")

plt.grid(alpha=0.3)

plt.show()

```

## Distribution Of Residuals

```

plt.figure(figsize=(8, 6))

sns.histplot(
    residuals,

```

```

        kde=True,
        color="green"
    )

plt.xlabel("Residual")
plt.ylabel("Frequency")

plt.title("Distribution of Residuals")

plt.show()

```

## Correlation Matrix

```

plt.figure(figsize=(10, 7))

sns.heatmap(
    df.corr(),
    annot=True,
    cmap="coolwarm",
    fmt=".2f"
)

plt.title("Correlation Matrix")

plt.show()

```

## Actual Vs Predicted Values

```

results = pd.DataFrame({
    "Actual BMI": y_test.values,
    "Predicted BMI": y_pred,
    "Residual": residuals.values
})

print("\n" + "=" * 60)
print("ACTUAL VS PREDICTED VALUES")
print("=" * 60)

print(results.head(15))

```

## Regression Equation

```

print("\n" + "=" * 60)
print("REGRESSION EQUATION")
print("=" * 60)

equation = f"BMI = {model.intercept_:.4f}"

for feature, coefficient in zip(X.columns, model.coef_):

```

```
equation += f" + ({coefficient:.4f} × {feature})"
print(equation)
```

## Interpretation

```
print("\n" + "=" * 60)
print("INTERPRETATION")
print("=" * 60)

print(f"""
1. MAE = {mae:.4f}

    The model's average absolute prediction error is
    approximately {mae:.2f} BMI units.

2. MSE = {mse:.4f}

    This represents the average squared prediction error.
    Larger errors receive greater penalty.

3. RMSE = {rmse:.4f}

    The typical prediction error is approximately
    {rmse:.2f} BMI units.

4. R2 = {r2:.4f}

    The model explains approximately
    {r2 * 100:.2f}% of the variation in BMI.

5. Residual Analysis:

    Residuals are calculated as Actual BMI - Predicted BMI.

    Ideally, residuals should be randomly distributed
    around zero.

6. Actual vs Predicted Plot:

    Points closer to the diagonal line indicate
    more accurate BMI predictions.

""")
```